



**Applying Machine Learning Models to Software Development
Multi-Agent Systems**

Multi-Agent LLM Configurations

Quantitative Evaluation in Software Engineering

Shahab Ali

ID 503271 — Group J4132

Semester 2 Progress Report Supervisor: Prof. Sergey Kovalchuk

ITMO University — FATII

Project Recap & Semester 2 Objective



- **Semester 1 (Theory):** Systematic Literature Review (PRISMA) of 19 SOTA papers → built a 7-layer agent role taxonomy.
- **Semester 2 (Practice):** Move from theory to experiments — implement agent roles and evaluate them quantitatively.
- **Goal:** Implement different roles in an agentic architecture (OpenHands) and benchmark them with quantitative comparison.
- **Research Questions:**
 - RQ1: What are the key agentic roles in LLM-based software engineering?
 - RQ2: How are these roles represented in LLM context?

From Theory to Practice: The Journey



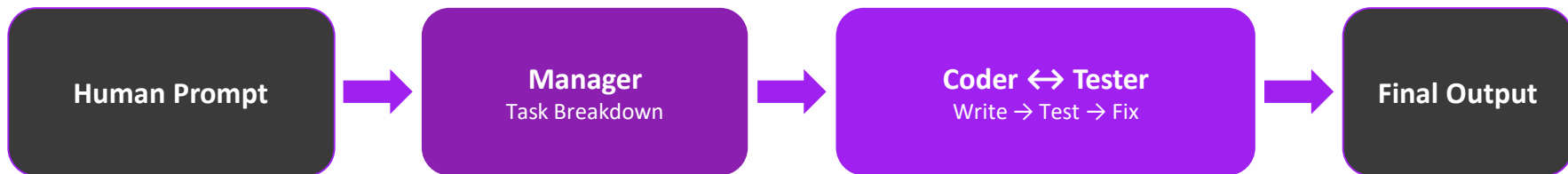
- ❖ **Weeks 1–2:** Designed 3-role architecture; tested local 7B model → failed (context collapse, safety refusal, prompt bleed).
- ❖ **Week 3:** Deployed Docker + Ollama (8B). Documented 3 failures: ReadTimeout, state corruption, schema brittleness.
- ❖ **Week 4:** Switched to Cloud 120B model → full Manager-Coder-Tester loop succeeded. Self-Healing first observed.
- ❖ **Weeks 5+:** Numerical evaluation on HumanEval and SWE-bench using the Cerebras 120B API.

The Three-Role Agent Taxonomy

- ❑ **Manager (Orchestration Layer):** Parses the user request into a step-by-step logic plan. Writes no code.
- ❑ **Coder (Implementation Layer):** Translates the plan into an executable Python script and saves it to the workspace.
- ❑ **Tester (Quality Assurance Layer):** Executes the code in the terminal, reads errors, and passes them back to the Coder for self-correction.
- ❑ **Key insight from SLR:** The QA layer dominates the literature (9 of 19 papers) — verification matters more than generation.

Architecture: The Multi-Agent Loop

- ❑ The workflow replaces human intervention during the implementation phase, built on the OpenHands CodeAct architecture.
- ❑ **Coder and Tester form a feedback loop** — code is written, executed, errors caught, and fixed autonomously until tests pass.
- ❑ Architecture goes beyond role prompting — it defines agent actions, observations, and memory.



Scientific Novelty

5 Original Contributions Not Found in Prior Literature:



➤ **Self-Healing Phenomenon**

First empirical documentation of autonomous error recovery in LLM agents
90% recovery rate (9/10 events)

➤ **Agentic Paralysis**

New failure mode identified and named agent knows next step but cannot execute tool call

➤ **Anti-Paralysis Prompt Engineering**

Single meta-cognitive instruction raises SWE-bench pass rate from 33.3% → 100%

➤ **Framework Brittleness vs Scale**

First controlled comparison proving model scale (not role) determines success 7B = 0%, 120B = 100%

➤ **10-Type Failure Taxonomy**

First structured classification of failure modes in OpenHands ,CodeAct agentic framework

21 experimental runs across
HumanEval + SWE-bench
4 phenomena named for the
first time
10 failure types documented

Experimental Setup & Configurations

- **Framework:** OpenHands CodeAct (Docker + Ubuntu/WSL). **Benchmarks:** HumanEval + SWE-bench.
- **Two variables isolated:** LLM Scale (7B local vs 120B cloud) and Agent Role (Single vs Multi-Agent). 21 total runs.

Config	LLM Scale	Agent Role	Setup
1	Local 7B	Single Agent	Coder only
2	Local 7B	Multi-Agent	Coder + Tester
3	Cloud 120B	Single Agent	Coder only
4	Cloud 120B	Multi-Agent	Coder + Tester

Phase 1: HumanEval Results (#1-15)



- ❑ Clear result: 7B models crash 100% of the time; 120B models pass 100% of tasks — scale is the deciding factor, not role.

LLM	Role	Tasks	Pass%	Crash%	Heal%
Local 7B	Single Coder	5	0%	100%	0%
Local 7B	Coder+Tester	5	0%	100%	0%
Cloud 120B	Single Coder	5	100%	0%	100%
Cloud 120B	Coder+Tester	10	100%	0%	100%

Discovery: The Self-Healing Phenomenon



- ❑ The 120B model autonomously recovered from framework errors that completely crashed the 7B model.
- ❑ **Errors self-healed:** missing security_risk parameter, file conflicts, placeholder hallucination, and minor reasoning loops.
- ❑ This is an emergent capability of large models — the agent reads its own error trace and fixes itself.

90%

Autonomous recovery rate

9 / 10

Error events self-healed

Phase 2: SWE-bench Results (#1-6)

- ❑ Real repository tasks are harder: 4 of 6 passed. Failures revealed Agentic Paralysis and environment-dependency limits.

Task	Files	Anti-Paralysis	Result	Finding
#1	calculator + app	No	PASS	Agentic Loop
#2	app.py	No	FAIL	Env Dependency
#3	calculator.py	No	FAIL	Chain Failure
#4	math_utils	Yes	PASS	Perfect 8/8
#5	humaneval_results	Yes	PASS	Perfect 11/11
#6	calculator ext.	Yes	PASS	Recovered

Agentic Paralysis & The Anti-Paralysis Fix



- **Agentic Paralysis:** the agent knows the next step but fails to execute the tool. Severe case: 200+ loops → token exhaustion → crash.
- **The Fix:** a single meta-cognitive instruction — “max 3 attempts per action; if stuck, run `ls /workspace/` to reorient.”

WITHOUT instruction

33.3%

SWE-bench pass rate

WITH instruction

100%

SWE-bench pass rate

Complete Failure Taxonomy: 10 Types

- ❑ Across 21 runs, ten distinct failure types were identified and documented — a key research contribution.

#	Failure Type	Recovery	#	Failure Type	Recovery
1	XML Schema Error	Self-Heal	6	Agentic Paralysis	None
2	Context Collapse	None	7	Verbose Paralysis	Partial
3	Prompt Bleed	None	8	Chain Failure	None
4	File Conflict	Pivot	9	Env Dependency	None
5	Placeholder Halluc.	Self-Heal	10	File Not Found	Explore

Key Findings & Overall Statistics



- **1. Framework Brittleness vs Scale:** model scale, not role, decides success (7B = 0%, 120B = 100%).
- **2. Self-Healing:** emergent 90% autonomous error recovery in the 120B model.
- **3. Agentic Paralysis:** a critical limitation of SOTA frameworks on multi-step tasks.
- **4. Anti-Paralysis Prompting:** one instruction raised SWE pass rate 33.3% → 100%.
- **5. Context Window Exhaustion:** CI/CD noise crashes agents; fixed with -q and head -n 20.

21

Total experimental runs

78.9%

120B model pass rate

10

Failure types documented

Conclusion & Next Steps

- **Conclusion:** The Semester 1 taxonomy was empirically validated. Model scale drives reliability, and the QA layer (self-correction) is central to multi-agent success.
- **Next Steps:**
 - Add a 3-role configuration (Manager + Coder + Tester) to the evaluation.
 - Expand the dataset with more HumanEval and SWE-bench tasks.
 - Publish the public repository and prepare the research preprint.

Thank you!

it's **MO** *re than a*
UNIVERSITY

<https://github.com/Shahaborakzai/multi-agent-llm-se-research>